



Explore | Expand | Enrich

<Codemithra />TM

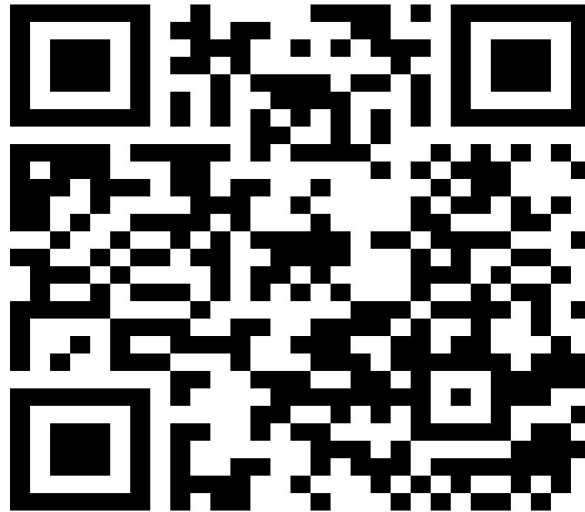
MANEUVERING PROBLEM



TEST TIME ON SORTED UNIQUE PERMUTATION

URL: <https://forms.gle/54ANJLeEKjZBG59B7>

QR CODE:



MANEUVERING PROBLEM

INTRODUCTION

- The problem is also called Maneuvering a Cave.
- You are given an $m \times n$ matrix as input.
- The task is to count all the possible paths from top left to bottom right of a $m \times n$ matrix.

Constraints:

1. You start from 0×0 and end at $m-1 \times n-1$.
2. From each cell you can either move only to right or down.



MANEUVERING PROBLEM

Input

$m = 2, n = 2$

□ There are two paths :

$(0, 0) \rightarrow (0, 1) \rightarrow (1, 1) ; (0, 0) \rightarrow (1, 0) \rightarrow (1, 1)$

$m = 2, n = 3$

□ There are three paths :

$(0, 0) \rightarrow (0, 1) \rightarrow (0, 2) \rightarrow (1, 2)$

$(0, 0) \rightarrow (0, 1) \rightarrow (1, 1) \rightarrow (1, 2)$

$(0, 0) \rightarrow (1, 0) \rightarrow (1, 1) \rightarrow (1, 2)$

Examples

Output

2

3



MANEUVERING PROBLEM

```
public class EthCode {  
    // Returns count of possible paths to reach  
    // cell at row number m and column number n  
    // from the topmost leftmost cell (cell at 1, 1)  
    static int numberOfPaths(int m, int n)  
    {  
        // If either given row number is first or  
        // given column number is first  
        if (m == 1 || n == 1)  
            return 1;  
  
        // If diagonal movements are allowed then  
        // the last addition is required.
```

```
        return numberOfPaths(m - 1, n) +  
            numberOfPaths(m, n - 1);  
        // + numberOfPaths(m-1, n-1);  
    }  
    public static void main(String args[])  
    {  
        System.out.println(numberOfPaths(3, 3));  
    }  
}
```



MANEUVERING PROBLEM

Solutions-1's Complexity

Time Complexity : Exponential Time Complexity



MANEUVERING PROBLEM

Like Divide and Conquer, DP combines solutions to sub-problems.

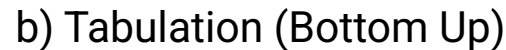
Overlapping Sub-problems :

- Dynamic Programming is mainly used when solutions of the same subproblems are needed again and again.
- Computed solutions to subproblems are stored in a table so that these don't have to be recomputed.
- So Dynamic Programming is not useful when there are no common (overlapping) subproblems because there is no point storing the solutions if they are not needed again.



Fibonacci Sequence - Space Complexity analysis

```
Fib(n)
{
    if n <= 2
        return n
    else
        return Fib(n-1) + Fib(n-2)
}
```



MANEUVERING PROBLEM

- The memoized program for a problem is similar to the recursive version with a small modification that looks into a lookup table before computing solutions. We initialize a lookup array with all initial values as NIL. Whenever we need the solution to a subproblem, we first look into the lookup table. If the precomputed value is there then we return that value, otherwise, we calculate the value and put the result in the lookup table so that it can be reused later.

MANEUVERING PROBLEM

- Tabulation (Bottom Up): The tabulated program for a given problem builds a table in bottom-up fashion and returns the last entry from the table. For example, for the same Fibonacci number, we first calculate $\text{fib}(0)$ then $\text{fib}(1)$ then $\text{fib}(2)$ then $\text{fib}(3)$, and so on. So literally, we are building the solutions of subproblems bottom-up.
- Both Tabulated and Memoized store the solutions of subproblems.
- In Memoized version, table is filled on demand while in the Tabulated version, starting from the first entry, all entries are filled one by one.
- Unlike the Tabulated version, all entries of the lookup table are not necessarily filled in Memoized version.

MANEUVERING PROBLEM

Optimal Substructure :

- A given problems has Optimal Substructure Property if optimal solution of the given problem can be obtained by using optimal solutions of its subproblems.

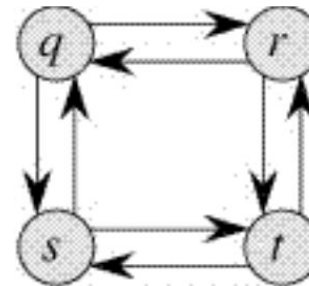
For example, the Shortest Path problem has following optimal substructure property :

- If a node x lies in the shortest path from a source node u to destination node v then the shortest path from u to v is combination of shortest path from u to x and shortest path from x to v . The standard All Pair Shortest Path algorithm like Floyd–Warshall and Single Source Shortest path algorithm for negative weight edges like Bellman–Ford are typical examples of Dynamic Programming.



MANEUVERING PROBLEM

- On the other hand, the Longest Path problem doesn't have the Optimal Substructure property. Here by Longest Path we mean longest simple path (path without cycle) between two nodes. CONSIDER THE FIGURE of the graph There are two longest paths from q to t : $q \rightarrow r \rightarrow t$ and $q \rightarrow s \rightarrow t$. Unlike shortest paths, these longest paths do not have the optimal substructure property. For example, the longest path $q \rightarrow r \rightarrow t$ is not a combination of longest path from q to r and longest path from r to t , because the longest path from q to r is $q \rightarrow s \rightarrow t \rightarrow r$ and the longest path from r to t is $r \rightarrow q \rightarrow s \rightarrow t$.



MANEUVERING PROBLEM

Optimal Substructure :

- An optimal solution to a problem contains within it an optimal solution to subproblems.
- Optimal Solution to the entire problem is build in a bottom-up manner from optimal solutions to subproblems.

Overlapping Subproblems :

- If a recursive algorithm revisits the same subproblems over and over => the problem has overlapping subproblems.



MANEUVERING PROBLEM

```
public class EthCode {  
    // Returns count of possible paths to reach  
    // cell at row number m and column number n from  
    // the topmost leftmost cell (cell at 1, 1)  
    static int numberOfPaths(int m, int n)  
    {  
        // Create a 2D table to store results  
        // of subproblems  
        int count[][] = new int[m][n];  
        // Count of paths to reach any cell in  
        // first column is 1  
        for (int i = 0; i < m; i++)  
            count[i][0] = 1;  
        // Count of paths to reach any cell in  
        // first column is 1  
        for (int j = 0; j < n; j++)  
            count[0][j] = 1;  
    }  
}
```



MANEUVERING PROBLEM

```
// Calculate count of paths for other
// cells in bottom-up manner using

// the recursive solution
for (int i = 1; i < m; i++) {
    for (int j = 1; j < n; j++)
        // By uncommenting the last part the
        // code calculates the total possible paths
        // if the diagonal Movements are allowed
        count[i][j] = count[i - 1][j] + count[i][j - 1]; //+
        count[i-1][j-1];
    }
    return count[m - 1][n - 1]; }
// Driver program to test above function
public static void main(String args[])
{ //Input the values for m and n
    int m = 3;
    int n = 3;
    System.out.println(numberOfPaths(m, n)); }}
```



MANEUVERING PROBLEM

Complexity

Time Complexity : $O(m \times n)$



MANEUVERING PROBLEM

```
public class EthCode {  
  
    static int numberOfPaths(int m, int n)  
    {  
        // Create a 1D array to store results of subproblems  
        int[] dp = new int[n];  
        dp[0] = 1;  
  
        for (int i = 0; i < m; i++) {  
            for (int j = 1; j < n; j++) {  
                dp[j] += dp[j - 1];  
            }  
        }  
  
        return dp[n - 1];  
    }  
}
```

```
public static void main(String args[])  
{  
    System.out.println(numberOfPaths(3, 3));  
}
```

MANEUVERING PROBLEM

Complexity

Time Complexity : $O(n)$

This is an optimization over the DP variant of the problem We will be using a 1D array here.



THANK YOU